TABLE OF CONTENTS

LIST OF FIGURES vi

LIST OF TABLES vii

LIST OF ABBREVIATIONS viii

CHAPTER 1. INTRODUCTION 1

CHAPTER 2. LITERATURE REVIEW 3

CHAPTER 3. PROPOSED SYSTEM 5

3.1 Problem Definition 5
3.2 Theory 5
3.2.1 Initialization 6
3.2.2 Round Function 6
3.2.2.1 Sub Cells 7
3.2.2.2 AddConstants 8
3.2.2.3 Add Round Tweakey 8
3.2.2.4 Shift Rows 9
3.2.2.5 Mix Columns 9
3.2.2.6 S-box implementation using logic 0ptimization based on truth table 10
3.3 Design 11
3.3.1 Algorithm 11
3.3.2 Software Design 13
3.3.2.1 Xilinx Vivado 13
3.3.3 Hardware design 14
3.3.3.1 AC701 Evaluation Board for the Artix-7 FPGA 14
3.4 Methodology 16
3.4.1 Encryption 16
3.4.2 Decryption 17

3.5 Work done 19
3.5.1 S-box design 19

CHAPTER 4. Result and discussion **27**

CHAPTER 5. CONCLUSION and future scope **31**
5.0.1 Conclusion 31
5.0.2 Future Scope 31

REFERENCES **33**

APPENDIX. VHDL Code **35**

APPENDIX. DATA SHEETS **58**

LIST OF FIGURES

3.1 The SKINNY round function applies five different transformations: Sub Cells (SC), Add Constants (AC), Add Round Tweakey (ART), Shift Rows (SR) and Mix Columns (MC). 7

3.2 The tweakey schedule in SKINNY. Each tweakey word TK1, TK2 and TK3 (if any) follows a similar transformation update, except that no LFSR is applied to TK1. 8

3.3 S-Box architecture using combinational logic 10

3.4 Algorithm Implementation 11

3.5 Algorithm Implementation 12

3.6 Xilinx 13

3.7 AC701 Board 15

3.8 Block Diagram for Encryption 16

3.9 Block Diagram for Decryption 17

4.1 Number of LUTs and Flip-Flops befor optimization 27

4.2 Number of LUTs and Flip-Flops after optimization 27

4.3 Power befor optimization 28

4.4 Power after optimization 28

4.5 Delay befor optimization 29

4.6 Delay after optimization 29

.1 AC701 Board Components 58

LIST OF TABLES

3.1 1s1 module 19

4.1 Pre Optimization and post optimization 30

.1 AC701 Board Component Descriptions 60

.2 AC701 Board FPGA Configuration Modes 60

LIST OF ABBREVIATIONS

AEAD Authenticated Encryption with Assosiated Data

AES Advanced Encryption Standard

ASIC Application-Specific Integrated Circuit

CMOS Complementary Metal-Oxide-Semiconductor

FPGA Feild Program Gate Array

HDL Hardware Description Language

ICACDOT International Conference on Automatic Control and Dynamic Optimization Techniques

IOT Internet Of Things

LFSR Linear Feedback Shift Register

UTHM Universiti Tun Hussein Onn Malaysia

CHAPTER 1

INTRODUCTION

As the demand for lightweight and resource-efficient cryptographic algorithms continues to rise in the era of Internet of Things (IoT) and edge computing, there is a growing need for secure and efficient solutions tailored to constrained environments. This project focuses on the implementation of the SKINNY-AEAD block cipher, a lightweight cryptographic algorithm designed for resource-constrained devices, on Field-Programmable Gate Arrays (FPGAs) to achieve optimal area efficiency.

The SKINNY-AEAD algorithm is chosen for its suitability in lightweight applications, offering a balance between security and computational efficiency. The project aims to design and implement a dedicated hardware architecture for SKINNY-AEAD on an FPGA platform, optimizing the use of available resources while ensuring the preservation of the algorithm's cryptographic properties.

Key objectives include the exploration of novel design techniques, to maximize the throughput of the cipher on FPGA devices. The project also investigates the integration of efficient key scheduling mechanisms to further enhance the overall performance of the SKINNY-AEAD block cipher. Additionally, the implementation will be rigorously tested for functionality, security, and performance metrics to validate its effectiveness in lightweight applications.

The outcome of this project is expected to contribute to the advancement of lightweight cryptography by providing an area-efficient hardware implementation of the SKINNY-AEAD block cipher suitable for deployment in resource-constrained environments. The results will be valuable for securing communication and data

integrity in lightweight devices, including those commonly found in IoT and edge computing scenarios.

CHAPTER 2

LITERATURE REVIEW

SKINNY AEAD and SKINNY HASH

Christof Beierle ,Jeremy Jean,Stefan K¨olbl, Gregor Leander, Amir Moradi,Thomas Peyrin, Yu Sasaki,Pascal Sasdrich,Siang Meng Sim

[1] This paper provides a detailed specification of Tweakable Block Ciphers SKINNY-128-256 and SKINNY-128-384, alongside the AEAD Scheme SKINNY-AEAD and the Hash Functionality SKINNY-Hash. The introduction outlines the purpose and background of the lightweight tweakable block ciphers in the SKINNY family. The document then delves into design rationale, security claims, parameter sets, and implementation considerations for SKINNY-AEAD and SKINNY-Hash. For SKINNY-AEAD, the authors utilize the ΘCB3 framework with SKINNY-128-384, ensuring provable security in the nonce-respecting setting. SKINNY-Hash employs a sponge construction based on SKINNY-128-256, claiming robust security against collision and preimage attacks. The paper details parameter sets for SKINNY-128-256 and SKINNY-128-384, providing insight into key size, block size, and rounds, with rationale and guidance for selection. The document concludes with implementation considerations, guiding secure and efficient implementation while addressing common pitfalls.

Design of AES S-Box using combinational logic optimization

Nabihah Ahmad,Warsuzarina Mat Jubadi Faculty of Electrical and Electronic Engineering Universiti Tun Hussein Onn Malaysia (UTHM) Johor, Malaysia

[2] This paper presents a novel approach to reducing the hardware complexity of AES S-Box (Substitution box), which is a crucial component of the AES encryption algorithm. The paper begins with an introduction to AES encryption and the importance of S-Box in the algorithm. It then describes the proposed combinational logic design of S-Box and how it reduces the overall delay and improves the efficiency of high-speed applications. The design optimization of S-Box is proposed in section IV, which includes the discussion of the truth table and Boolean simplification. The paper also provides a detailed description of the architecture of the proposed design, including the combinational logic functions, the D gate, NOT gate, and Boolean expressions for the output of Module 1(M1). The results of the design optimization are presented in section V, which includes the comparison and evaluation with previous work. The paper concludes with a summary of the proposed approach and its practical implications for FPGA implementation in terms of gate area, hardware, total area, and delay. Overall, this paper provides a comprehensive overview of the design of AES S-Box using combinational logic optimization and its potential applications in high-speed encryption systems.

Logic optimization of AES S-Box

S Raghuram, M Chennakrishna, AS Varun Karthik

[3] 2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICACDOT), 1042-1046, 2016

This paper presents the implementation of Rijndael S-Box using combinational logic for the SubByte transformation in the Advanced Encryption Standard algorithm for ASIC. The main focus of this project is on achieving reduction in area occupancy and power consumption for the S-box module. We have realized an optimized implementation of the S-box in the Verilog HDL, and have subsequently synthesized it using 90nm CMOS standard cell library. Using a novel method of design reuse in the S-Box logic, the optimized design implemented, consumes 146 cells and the total power dissipation of 32,911 nW which is significantly lower.

CHAPTER 3

PROPOSED SYSTEM

3.1 Problem Definition

The task at hand involves refining the implementation of cryptographic algorithms on FPGA, focusing specifically on SKINNY ciphers. The overarching goals encompass optimizing FPGA area usage by capitalizing on the unique characteristics of SKINNY ciphers, thereby reducing footprint without compromising functionality. Simultaneously, the objective is to improve computational efficiency and resource utilization for enhanced overall system efficiency [5] . Furthermore, there is a critical imperative to bolster the security aspects of the cryptographic system, ensuring compliance with or surpassing established security standards while employing SKINNY ciphers. This comprehensive approach aims to provide a solution that not only addresses current inefficiencies in FPGA area utilization and computational efficiency but also strengthens the security framework of the deployed cryptographic algorithms

3.2 Theory

It provides a system model for the SKINNY family of lightweight tweakable block ciphers, including SKINNY-128-256 and SKINNY-128-384. The system model describes the environment in which the ciphers are used, including the attacker model, the security goals, and the assumptions made about the system.

The attacker model assumes that the attacker has access to the ciphertext and can perform chosen plaintext and chosen ciphertext attacks.[1] The security goals

include confidentiality, integrity, and authenticity, and the system assumes that the attacker cannot break these goals with a probability greater than a certain threshold.

The system model also includes information on the parameters used in the ciphers, including the key size, block size, and number of rounds. The authors explain the rationale behind their choices for these parameters and provide some guidance on how to choose appropriate parameter sets for different applications.

Overall, the system model provides a framework for understanding the security properties of the SKINNY family of ciphers and for evaluating their suitability for different applications.

3.2.1 Initialization

The ciphers receive a plaintext $m = m_0 \parallel m_1 \parallel \ldots \parallel m_{13} \parallel m_{14} \parallel m_{15} \parallel$, where the m_i are 8-bit words. The initialization of the ciphers' internal state are performed by simply setting $IS_i = m_i$ for $0 \leq i \leq 15$, i.e.,

$$IS = \begin{bmatrix} m_0 & m_1 & m_2 & m_3 \\ m_4 & m_5 & m_6 & m_7 \\ m_8 & m_9 & m_{10} & m_{11} \\ m_{12} & m_{13} & m_{14} & m_{15} \end{bmatrix} \tag{3.1}$$

The state is loaded in a row-wise manner , in contrast to the column-wise method used, for instance, in AES. This represents a choice that prioritizes hardware-friendliness[1].

The ciphers receive a tweakey inputs are also loaded row-wise.

3.2.2 Round Function

A single encryption round in SKINNY consists of five operations [1] performed in the following sequence: Sub Cells, Add Constants, Add Round Tweakey, Shift Rows, and Mix Columns as shown in Fig. 3.1.The number of rounds r executed

during encryption is contingent on the size of the tweakey. Specifically, SKINNY-128-256 utilizes $r = 48$ rounds, while SKINNY-128-384 employs $r = 56$ rounds. It's important to note that no whitening key is utilized in this process[2].

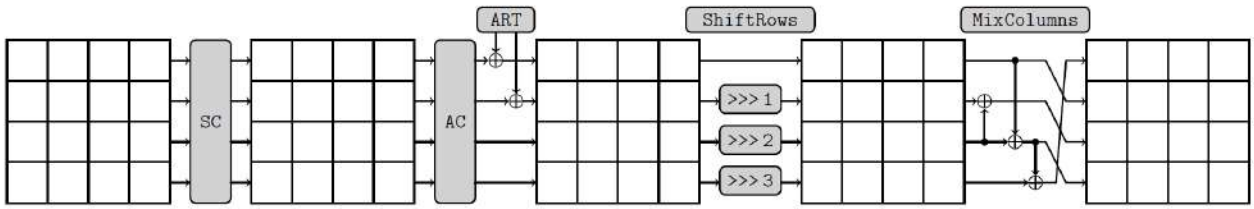

Fig. 3.1. The SKINNY round function applies five different transformations: Sub Cells (SC), Add Constants (AC), Add Round Tweakey (ART), Shift Rows (SR) and Mix Columns (MC).

3.2.2.1 Sub Cells

An S-box, or substitution box, is a fundamental component in symmetric key cryptography, particularly in block ciphers. It is responsible for substituting one set of values with another during the encryption or decryption process. S-boxes introduce non-linearity into the cryptographic algorithm, enhancing its resistance to various attacks. S-box is a crucial component in modern block ciphers, contributing to their security by introducing non-linearity and confusion in the encryption process. The design and properties of the S-box are carefully considered to ensure the overall strength of the cryptographic algorithm.

An 8-bit Sbox S_8 is applied to every cell of the ciphers internal state.If $x_0, \ldots, x_7$ represent the eight inputs bits of the Sbox (x_0 being the least signiffcant bit), it basically applies the below transformation on the 8-bit state:

$$(x_7, x_6, x_5, x_4, x_3, x_2, x_1, x_0) \rightarrow (x_7, x_6, x_5, x_4 \oplus \overline{(x_7 \vee x_6)}, x_3, x_2, x_1, x_0 \oplus \overline{(x_3 \vee x_2)}); \tag{3.2}$$

followed by the bit permutation

$$(x_7, x_6, x_5, x_4, x_3, x_2, x_1, x_0) \rightarrow (x_2, x_1, x_7, x_6, x_4, x_0, x_3, x_5)$$

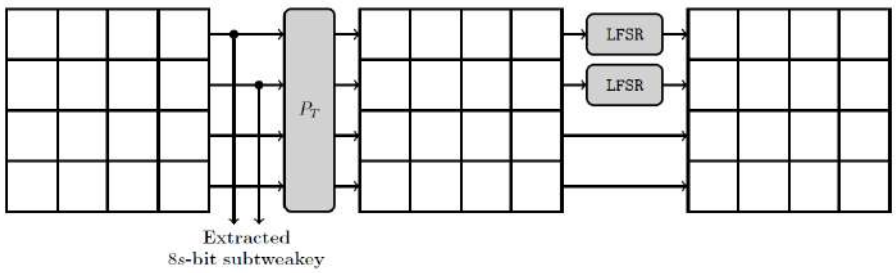

Fig. 3.2. The tweakey schedule in SKINNY. Each tweakey word TK1, TK2 and TK3 (if any) follows a similar transformation update, except that no LFSR is applied to TK1.

repeating this process four times, except for the last iteration where there is just a bit swap between S_1 and S_2

3.2.2.2 AddConstants

A 6-bit affine LFSR, whose state is denoted $(rc_5, rc_4, rc_3, rc_2, rc_1, rc_0)$ is used to generate round constants. Its update function is defined as:

$$(rc_5 \parallel rc_4 \parallel rc_3 \parallel rc_2 \parallel rc_1 \parallel rc_0) \rightarrow (rc_4 \parallel rc_3 \parallel rc_2 \parallel rc_1 \parallel rc_0 \parallel rc_5 \oplus rc_4 \oplus 1)$$

The six bits are set to zero initially and undergo updates before their application in a specific round. The LFSR bits are organized into a 4x4 array, with only the first column of the state being influenced by these LFSR bits[10].The round constants are combined with the state, respecting array positioning, using bitwise exclusive-or.

3.2.2.3 Add Round Tweakey

The first and second rows of all tweakey arrays are extracted and bitwise exclusive-ored to the ciphers internal state, respecting the array positioning

More formally, for $i = 0, 1$ and $j = 0, 1, 2, 3$, we have

- $IS_{i,j} = IS_{i,j} \oplus TK1_{i,j} \oplus TK2_{i,j}$ for SKINNY-128-256
- $IS_{i,j} = IS_{i,j} \oplus TK1_{i,j} \oplus TK2_{i,j} \oplus TK3_{i,j}$ for SKINNY-128-384

The tweakey arrays are updated as follows, a permutation is applied on the cells positions of all tweakey,and every cell of the first and second rows of TK2 (resp., TK2 and TK3) are individually updated with an LFSR arrays

3.2.2.4 Shift Rows

Similar to the AES(Addvanced Encryption Standard), in this layer, the rows of the cipher state cell array undergo rotation. Specifically, the second, third, and fourth cell rows are rotated by 1, 2, and 3 positions to the right, respectively.permutation P is applied on the cells positions of the cipher internal state cell array: for all $0 \leq i \leq 15$ we set $IS_i \leftarrow IS_{P[i]}$ with

$P = [0,1,2,3,7,4,5,6,10,11,8,9,13,14,15,12]$.

3.2.2.5 Mix Columns

Each column of the cipher internal state array is multiplied by the following binary matrix M

$$M = \begin{pmatrix} 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \end{pmatrix} \tag{3.3}$$

The resulting configuration of the internal state array determines the ciphertext, with the cells[6] being unpacked in a manner similar to the packing process during initialization. The decryption process closely resembles encryption since all cipher elements have straightforward inverses. Specifically, components like Sub Cells and Mix Columns, based on a generalized Feistel structure [10] , have easily deducible inverses that can be implemented with the same number of operations[3].

3.2.2.6 S-box implementation using logic 0ptimization based on truth table

This design tackles the slowness (delay) caused by traditional lookup tables in S-boxes [2] . It uses a combination of basic logic circuits (combinational logic) and special math (composite field arithmetic) to speed things up.

The proposed S-Box architecture is illustrated in Figure.

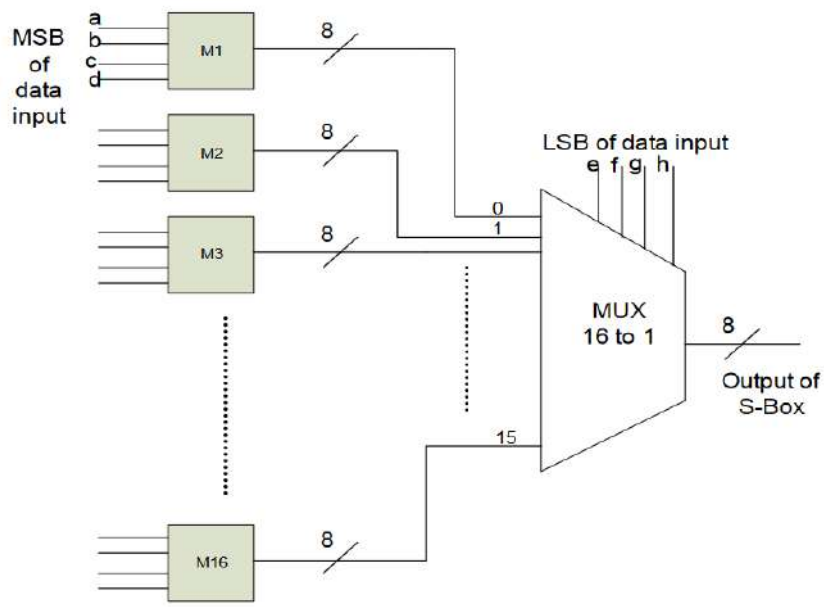

Fig. 3.3. S-Box architecture using combinational logic

The S-box works on 8-bit chunks of data. The first half (most significant bits) goes through a series of simplified logic functions (M1 to M16) designed to be very fast. These functions were created using a technique called a Karnaugh map. The other half (least significant bits) controls a switch (16-to-1 multiplexer) that picks the final output based on the results from the first half.

Overall, this design is intended to be used for the SubByte transformation step in encryption and is focused on improving speed.

3.3 Design

3.3.1 Algorithm

The software evaluation is performed based on bit-slice implementation.7 where the original S-box is replaced by the SKINNY-like ones.For 224 000 SKINNY-like S-boxes, they have the same software performance since they all contain the same number of AND and XOR instructions.

The Algorithm used for the Implementation [1] is as shown in the figure:

Algorithm 9 The authenticated encryption algorithm $\texttt{SKINNY-AEAD-M5-Enc}(K, N, A, M)$
In: 128-bit key K, 96-bit nonce N, associated data A, message M (both arbitrarily long)
Out: (C, tag), where C is the ciphertext with $|C| = |M|$ and tag is a 128-bit tag

```
//Associated data processing
A_0||A_1||...||A_{ℓ_a-1}||A_{ℓ_a} ← A with |A_i| = 128 for i ∈ {0,...,ℓ_a − 1} and |A_{ℓ_a}| < 128
Auth ← 0^128
LFSR ← 0^23||1
for all i = 0,...,ℓ_a − 1 do
    Auth ← Auth ⊕ SKINNY-128-256_{rev_24(LFSR)||00010010||N||K}(A_i)
    LFSR ← upd_24(LFSR)
if A_{ℓ_a} ≠ ϵ then
    Auth ← Auth ⊕ SKINNY-128-256_{rev_24(LFSR)||00010011||N||K}(pad10*(A_{ℓ_a}))
//Encryption
M_0||M_1||...||M_{ℓ_m-1}||M_{ℓ_m} ← M with |M_i| = 128 for i ∈ {0,...,ℓ_m − 1} and |M_{ℓ_m}| < 128
Σ ← 0^128
LFSR ← 0^23||1
for all i = 0,...,ℓ_m − 1 do
    C_i ← SKINNY-128-256_{rev_24(LFSR)||00010000||N||K}(M_i)
    Σ ← Σ ⊕ M_i
    LFSR ← upd_24(LFSR)
if M_{ℓ_m} = ϵ then
    C_{ℓ_m} ← ϵ
    T ← SKINNY-128-256_{rev_24(LFSR)||00010100||N||K}(Σ)
else
    R ← SKINNY-128-256_{rev_24(LFSR)||00010001||N||K}(0^128)
    C_{ℓ_m} ← M_{ℓ_m} ⊕ trunc_{|M_{ℓ_m}|}(R)
    LFSR ← upd_24(LFSR)
    Σ ← Σ ⊕ pad10*(M_{ℓ_m})
    T ← SKINNY-128-256_{rev_24(LFSR)||00010101||N||K}(Σ)
C ← C_0||C_1||...||C_{ℓ_m-1}||C_{ℓ_m}
//Tag generation
tag ← T ⊕ Auth
return (C, tag)
```

Fig. 3.4. Algorithm Implementation

Algorithm 11 The authenticated encryption algorithm $\texttt{SKINNY-AEAD-M6-Enc}(K, N, A, M)$
In: 128-bit key K, 96-bit nonce N, associated data A, message M (both arbitrarily long)
Out: (C, tag), where C is the ciphertext with $|C| = |M|$ and tag is a 64-bit tag

```
//Associated data processing
A_0||A_1||...||A_{ℓ_a-1}||A_{ℓ_a} ← A with |A_i| = 128 for i ∈ {0,...,ℓ_a − 1} and |A_{ℓ_a}| < 128
Auth ← 0^128
LFSR ← 0^23||1
for all i = 0,...,ℓ_a − 1 do
    Auth ← Auth ⊕ SKINNY-128-256_{rev_24(LFSR)||00011010||N||K}(A_i)
    LFSR ← upd_24(LFSR)
if A_{ℓ_a} ≠ ε then
    Auth ← Auth ⊕ SKINNY-128-256_{rev_24(LFSR)||00011011||N||K}(pad10*(A_{ℓ_a}))
//Encryption
M_0||M_1||...||M_{ℓ_m-1}||M_{ℓ_m} ← M with |M_i| = 128 for i ∈ {0,...,ℓ_m − 1} and |M_{ℓ_m}| < 128
Σ ← 0^128
LFSR ← 0^23||1
for all i = 0,...,ℓ_m − 1 do
    C_i ← SKINNY-128-256_{rev_24(LFSR)||00011000||N||K}(M_i)
    Σ ← Σ ⊕ M_i
    LFSR ← upd_24(LFSR)
if M_{ℓ_m} = ε then
    C_{ℓ_m} ← ε
    T ← SKINNY-128-256_{rev_24(LFSR)||00011100||N||K}(Σ)
else
    R ← SKINNY-128-256_{rev_24(LFSR)||00011001||N||K}(0^128)
    C_{ℓ_m} ← M_{ℓ_m} ⊕ trunc_{|M_{ℓ_m}|}(R)
    LFSR ← upd_24(LFSR)
    Σ ← Σ ⊕ pad10*(M_{ℓ_m})
    T ← SKINNY-128-256_{rev_24(LFSR)||00011101||N||K}(Σ)
C ← C_0||C_1||...||C_{ℓ_m-1}||C_{ℓ_m}
//Tag generation
tag ← trunc_64(T ⊕ Auth)
return (C, tag)
```

Fig. 3.5. Algorithm Implementation

3.3.2 Software Design

The software utilized and its operational functionalities are elucidated below.:

3.3.2.1 Xilinx Vivado

Vivado is a software suite for synthesis and analysis of hardware description language (HDL) designs, superseding Xilinx ISE with additional features for system on a chip development and high-level synthesis.Vivado represents a ground-up rewrite and re-thinking of the entire design flow (compared to ISE).

Fig. 3.6. Xilinx

The Vivado High-Level Synthesis compiler enables C, C++ and SystemC programs to be directly targeted into Xilinx devices without the need to manually create RTL. Vivado HLS is widely reviewed to increase developer productivity, and is confirmed to support C++ classes, templates, functions and operator overloading. Vivado 2014.1 introduced support for automatically converting OpenCL kernels to IP for Xilinx devices.OpenCL kernels are programs that execute across various CPU, GPU and FPGA platforms.

• Circuit Description:

Start with a description of the digital circuit using a hardware description language (HDL) such as Verilog or VHDL. Define the logic gates, flip-flops, and other

components of the circuit.

• Fault Model:

Identify the faults in the circuit. Common faults include stuck-at faults.

• Vivado Simulation:

Use Vivado's simulation features to simulate the circuit. In this step within the circuit description , the particular stuck at faults are induced by setting that particular point either permanently zero or permanently one.

• Comparing Results:

Monitor the outputs of the DUT and compare them against expected results and verifying the generated test pattern.

3.3.3 Hardware design

The hardware implementation done in AC701 Evaluation Board for the Artix-7 FPGA.

3.3.3.1 AC701 Evaluation Board for the Artix-7 FPGA

The AC701 evaluation board for the Artix®-7 FPGA provides a hardware environment for developing and evaluating designs targeting the Artix-7 XC7A200T-2FBG676C FPGA. The AC701 board provides features common to many embedded processing systems, including a 1 GB DDR3 SODIMM memory,a 256 Mb quad serial peripheral interface (quad SPI) flash memory,a secure Digital (SD) connector an 4-lane PCI Express® interface,both Fixed 200 MHz LVDS oscillator and I2C programmable LVDS oscillator with SMA connectors, a 10/100/1,000 tri-mode Ethernet PHY,a USB-to-UART bridge, GTP transceivers, general purpose I/ O,a PMBus voltage and current monitoring through TI power controller, and a UART interface. Other features can be added by using an FPGA mezzanine card (FMC) attached to the VITA-57 FPGA mezzanine connector provided on the board. A high pin count

(HPC) FMC connector is provided.

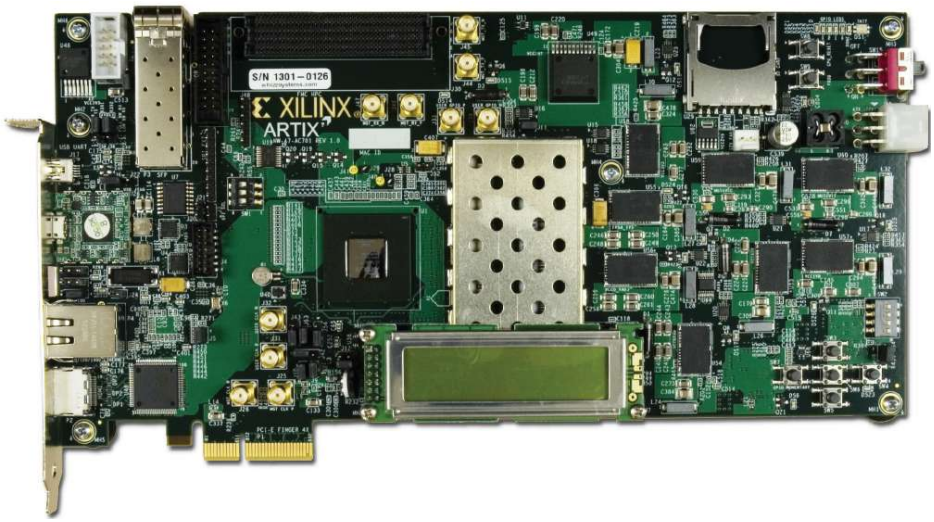

Fig. 3.7. AC701 Board

The Artix-7 FPGA, manufactured by Xilinx, is a popular choice for a wide range of applications due to its balance of performance, power efficiency, and cost-effectiveness. Artix-7 FPGAs are commonly used in DSP applications such as filtering, image processing, audio processing, and radar systems.They offer high-speed processing capabilities with dedicated DSP slices optimized for multiply-accumulate operations. Artix-7 FPGAs are used in various communication systems including wireless communication, wired networking, and software-defined radios.They support high-speed serial transceivers (up to 6.6 Gbps) for implementing protocols like PCIe, Ethernet, USB, and more. Artix-7 FPGAs can be used in embedded systems for implementing custom hardware accelerators, protocol converters, and system-on-chip (SoC) designs.They support embedded processing cores such as ARM Cortex-A9 or MicroBlaze, allowing for the integration of both hardware and software components on a single chip.Artix-7 FPGAs are employed in HPC applications such as data acceleration, encryption/decryption, and parallel processing.They offer high throughput and parallelism, making them suitable for tasks requiring massive computational power.Artix-7 FPGAs are used in test and measurement equipment for tasks such as signal generation, protocol analysis, and high-speed data acquisition.They offer flexible I/O configurations and high-speed interfaces for inter-

facing with external test equipment and devices.Artix-7 FPGAs are commonly used in educational settings and research projects for teaching digital design concepts, prototyping new algorithms, and exploring FPGA-based solutions.They provide a cost-effective platform for learning and experimentation with hardware design and implementation.Overall, the Artix-7 FPGA provides a versatile platform for implementing a wide range of digital designs, from simple logic functions to complex system-level applications, making it a popular choice across various industries and domains.

3.4 Methodology

3.4.1 Encryption

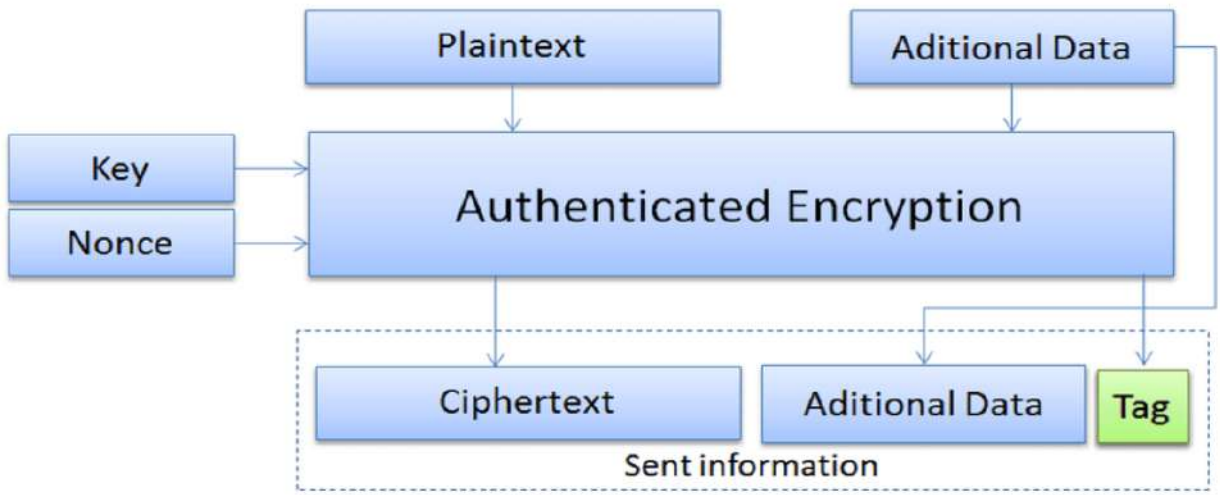

Fig. 3.8. Block Diagram for Encryption

AEAD (Authenticated Encryption with Associated Data) is a cryptographic process that provides both encryption and integrity verification of data in a single operation. In this process, a secret key is utilized to encrypt plaintext data, ensuring its confidentiality, while also generating an authentication tag that serves as a cryptographic checksum. The nonce (number used once) adds variability to the encryption process, preventing attackers from deriving patterns in encrypted data[4]. Associated data, such as metadata or headers, can be included in the encryption process without

being encrypted itself, offering additional context to the ciphertext. The combination of key, nonce, plaintext, and associated data contributes to the creation of a secure ciphertext. The authentication tag, computed based on these components, provides a means to verify the integrity of the decrypted data and ensures that it has not been tampered with during transmission. AEAD algorithms play a crucial role in securing communications and data storage, offering a comprehensive solution for confidentiality, integrity, and authenticity in cryptographic applications. Fig. 3.3 shows block diagram of encryption[9].

3.4.2 Decryption

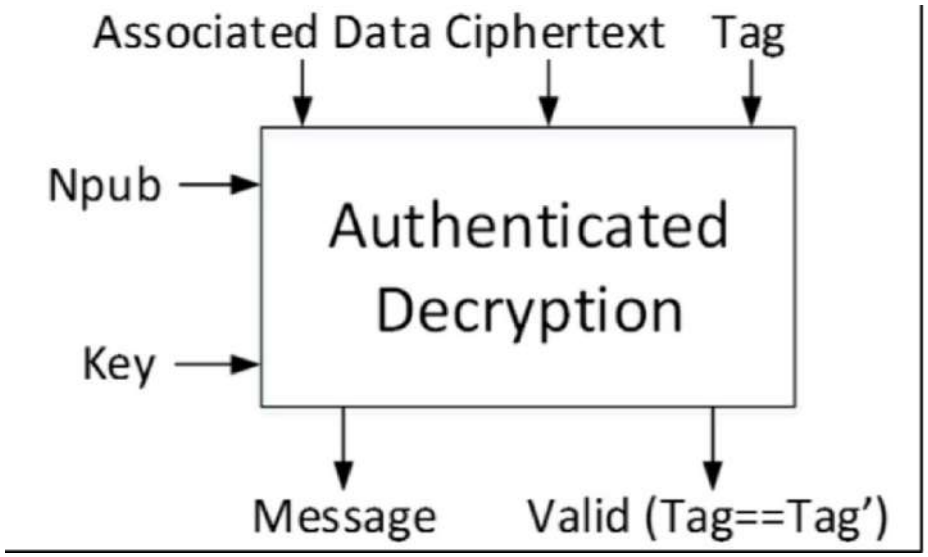

Fig. 3.9. Block Diagram for Decryption

In the AEAD (Authenticated Encryption with Associated Data) decryption process in cryptography, the key components include the key, nonce, associated data, plaintext, and the authentication tag. The decryption begins by extracting the nonce and authentication tag from the received ciphertext. The nonce is a unique value used to ensure that the same key can be safely reused for different messages. The associated data, which is additional information not encrypted but authenticated, is combined with the nonce and key to derive an authentication tag during encryption. The decryption process then utilizes the key and nonce to regenerate the authentication tag[8]. If the regenerated tag matches the received authentication tag, this

indicates both the integrity and authenticity of the associated data and ciphertext. Subsequently, the ciphertext is decrypted using the key and nonce to reveal the original plaintext. The associated data remains authenticated but is not part of the decrypted output. This process ensures the confidentiality, integrity, and authenticity of the transmitted data, providing a secure mechanism for encrypted communication in various cryptographic applications.Fig. 3.4[2] shows block diagram of decryption.

3.5 Work done

3.5.1 S-box design

a	b	c	d	e	f	g	h	1st row	y7	y6	y5	y4	y3	y2	y1	y0
0	0	0	0	0	0	0	0	00 - 65	0	1	1	0	0	1	0	1
0	0	0	0	0	0	0	1	01 - 4c	0	1	0	0	1	1	0	0
0	0	0	0	0	0	1	0	02 - 6a	0	1	1	0	1	0	1	0
0	0	0	0	0	0	1	1	03 - 42	0	1	0	0	0	0	1	0
0	0	0	0	0	1	0	0	04 - 4b	0	1	0	0	1	0	1	1
0	0	0	0	0	1	0	1	05 - 63	0	1	1	0	1	0	1	0
0	0	0	0	0	1	1	0	06 - 43	0	1	0	0	0	0	1	1
0	0	0	0	0	1	1	1	07 - 6b	0	1	1	0	1	0	1	1
0	0	0	0	1	0	0	0	08 - 55	0	1	0	1	0	1	0	1
0	0	0	0	1	0	0	1	09 - 75	0	1	1	1	0	1	0	1
0	0	0	0	1	0	1	0	0a - 5a	0	1	0	1	1	0	1	0
0	0	0	0	1	0	1	1	0b - 7a	0	1	1	1	1	0	1	0
0	0	0	0	1	1	0	0	0c - 53	0	1	0	1	0	0	1	1
0	0	0	0	1	1	0	1	0d - 73	0	1	1	1	0	0	1	1
0	0	0	0	1	1	1	0	0e - 5b	0	1	0	1	1	0	1	1
0	0	0	0	1	1	1	1	0f - 7b	0	1	1	1	1	0	1	1

Table 3.1. 1s1 module

boolean equations:-

$k7 = 0$

$k6 = 1$

$k5 = e'f'h' + fh + eh'$

$k4 = e$

$k3 = e'h'g'h + f'gh' + e'fg'h' + fgh$

$k2 = f'g'$

$k1 = g + f$

$k0 = g'h' + f + eg'$

Similarly, we can get other 15 modules Boolean equations

Module 2

$k7 = e'f'h + fh' + eh'$

$k6 = 0$

$k5 = e'f'h' + fh' + eh$

$k4 = e'f'h' + fh + eh$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = e'f'gh' + fh + egh$

$k0 = e'gh + efg + e'g'h' + fh + ef'g'$

Module 3

$k7 = 1$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = 0$

$k0 = e'gh + e'g'h' + ef'g' + efg$

Module 4

$k7 = e'f'h' + fh + eh$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h + e'fh' + eh$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = e'f'gh + fh' + egh'$

$k0 = g'h' + fg + ef'g'$

Module 5

$k7 = e'f'h + fh' + eh'$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h' + e'fh + eh'$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = e'f'g'h' + ef'g'h$

$k0 = e'gh + e'fg'h' + ef'g'h' + efg$

Module 6

$k7 = 0$

$k6 = e + f'g' + gh' + fh$

$k5 = eh + e'f'h' + gh + e'fg'$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = f'g'$

$k0 = e'gh + e'fg'h' + efg$

Module 7

$k7 = e'f'g'h' + fh + eh$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h + e'fh' + eh$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = e'f'g'h + e'f'g'h$

$k0 = e'g'h' + e'gh + ef'g'h' + efg$

Module 8

$k7 = 1$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g + fg' + gh + eg$

$k1 = 0$

$k0 = e'gh + e'g'h' + ef'g' + efg$

Module 9

$k7 = e'f'h + fh' + eh'$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h' + e'fh + eh'$

$k3 = e'f'g'h + f'gh' + e'f'gh' + fgh + eg$

$k2 = f'g'$

$k1 = e'f'h + fh' + f'g' + eh'$

$k0 = fh' + fg$

Module 10

$k7 = 0$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = f'g'$

$k0 = e'fg'h' + e'gh + ef'g$

Module 11

$k7 = e'f'h' + fh + eh$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h + e'fh' + eh$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = e'f'h' + fh + f'g' + eh$

$k0 = e'f'g + e'fg' + fh + efg$

Module 12

$k7 = 1$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = f'g'$

$k1 = o$

$k0 = f$

Module 13

$k7 = e'f'h + fh' + eh'$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h' + e'fh + eh'$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = e'h + g + f + eh'$

$k0 = e'g'h' + f + eg'h$

Module 14

$k7 = 0$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = g + f$

$k0 = g'h' + f + e'g'$

Module 15

$k7 = e'f'h' + fh + eh$

$k6 = 0$

$k5 = e'f'h' + fh + eh$

$k4 = f'h + e'fh' + eh$

$k3 = e'f'g'h + f'g'h' + e'fg'h' + fgh + eg$

$k2 = g + f$

$k1 = e'h' + g + f + eh$

$k0 = f + eg'h'$

Module 16

$k7 = 1$

$k6 = 1$

$k5 = e'f'h' + fh + eh$

$k4 = e$

$k3 = e'f'g'h + f'gh' + e'f'g'h' + fgh + eg$

$k2 = g + f$

$k1 = 1$

$k0 = f$

CHAPTER 4

RESULT AND DISCUSSION

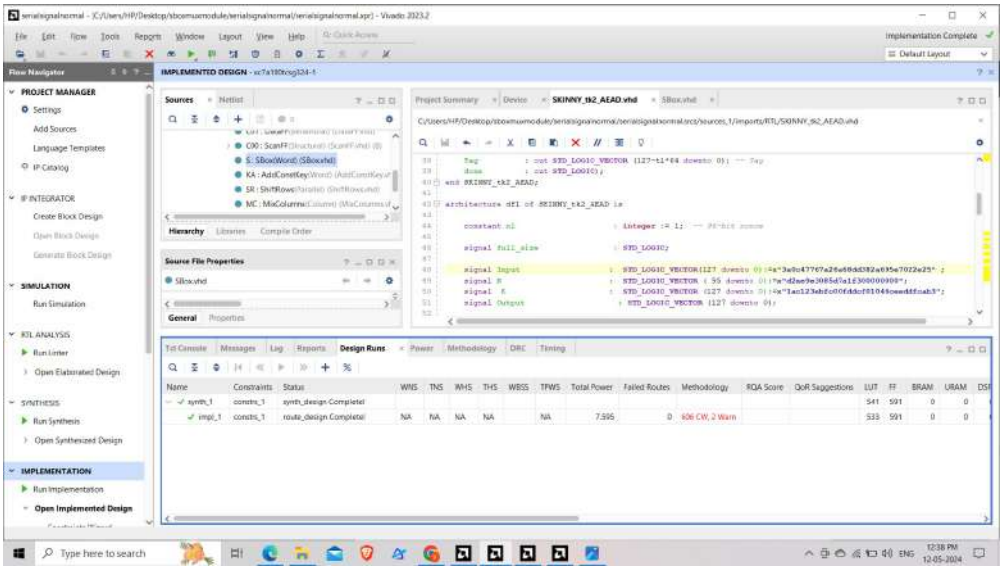

Fig. 4.1. Number of LUTs and Flip-Flops befor optimization

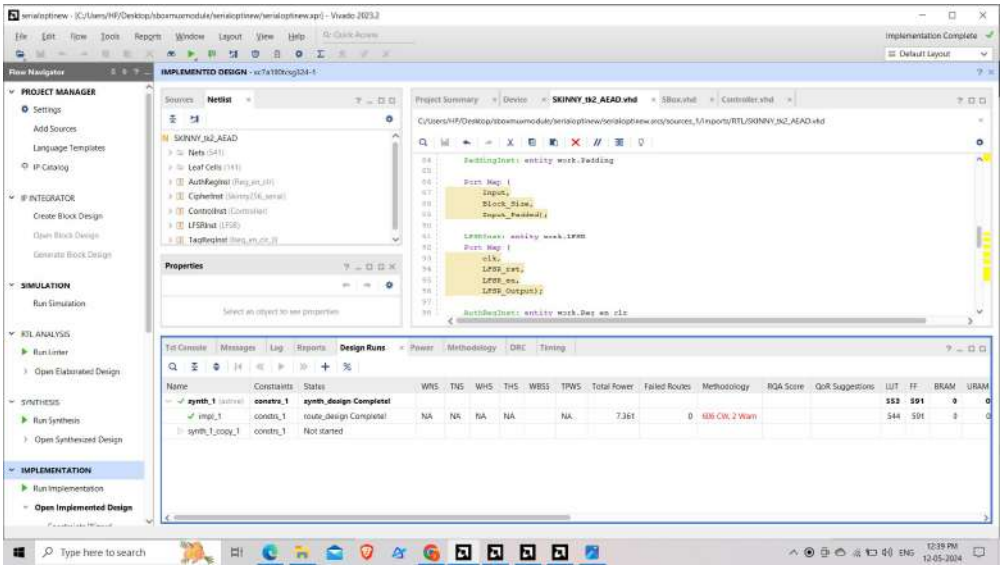

Fig. 4.2. Number of LUTs and Flip-Flops after optimization

In Fig 4.1 and Fig 4.2, the number of Look-Up Tables (LUTs) and Flip-Flops

(FFs) after and before optimization are depicted. These figures provide a visual representation of the impact of optimization techniques on the utilization of resources in the design. By comparing the figures, one can observe the changes in resource consumption, helping to assess the effectiveness of the optimization process.

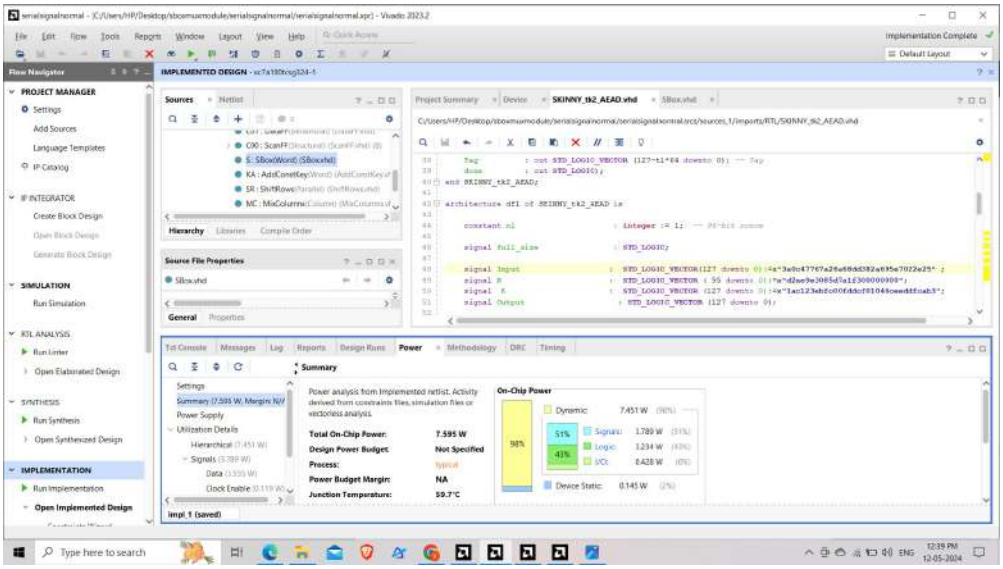

Fig. 4.3. Power befor optimization

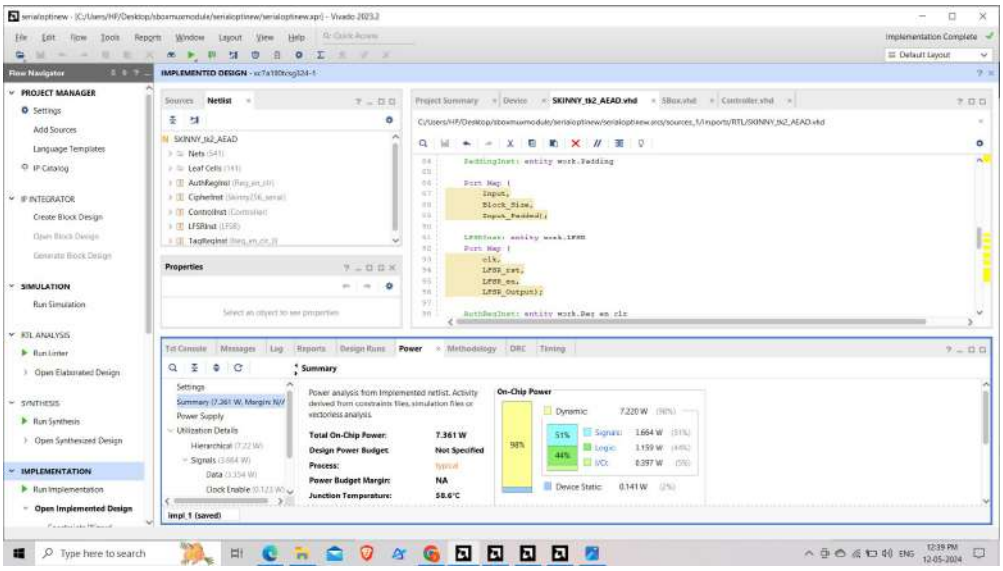

Fig. 4.4. Power after optimization

Fig 4.3 likely represents the power consumption before optimization, displaying higher power usage. This could be due to inefficiencies, such as excessive processing or inefficient hardware utilization.On the other hand Fig 4.4 illustrates the

power consumption after optimization measures have been implemented. Here, you would expect to see a reduction in power consumption compared to Fig 4.3.

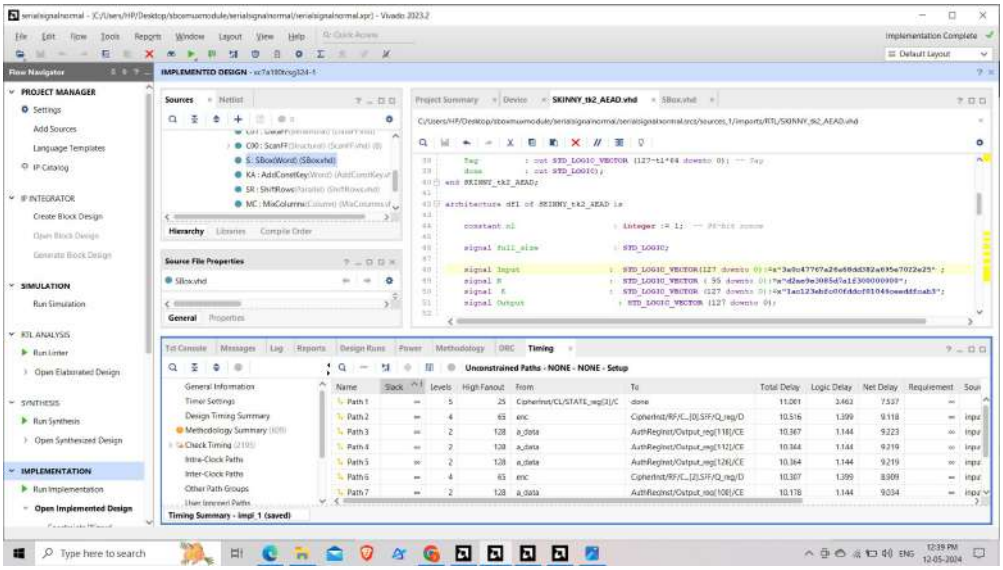

Fig. 4.5. Delay befor optimization

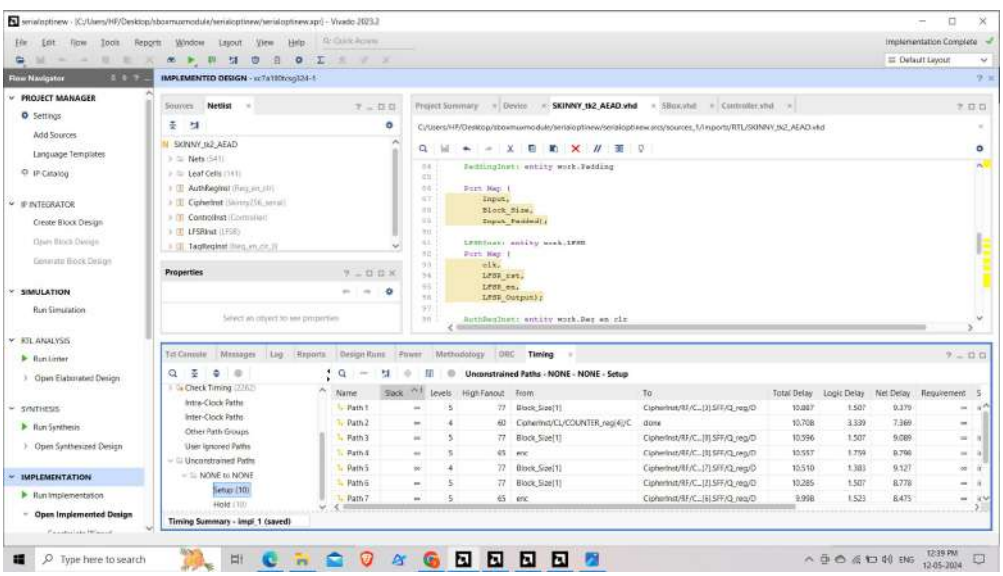

Fig. 4.6. Delay after optimization

Fig 4.5 and Fig 4.6 depict the reduction in delay before and after optimization, respectively. These figures likely showcase how certain optimizations or improvements have led to a decrease in the time it takes for a process or system to complete. ”Before optimization” would represent the initial delay experienced, while ”after optimization” demonstrates the decreased delay achieved through various enhancements.

In the serial unprotected method, the optimization primarily focused on enhancing delay and power performance, resulting in a slight improvement of delay from 11.001 ns to 10.187 ns and the power improvment from 3.234W to 3.159W. However, other key parameters experienced notable changes following the optimization process. While the delay and power showed a modest enhancement, the number of Look-Up Tables (LUTs) exhibited increase from 533 to 544 post-optimization and the flipflops remained the same,ie 591. This suggests a complex interplay between different design considerations, where improvements in one aspect may come at the expense of others.This detailed analysis aids in understanding how the design modifications contribute to enhancing performance, reducing resource usage, and optimizing the overall functionality of the system.

Properties	**Before**	**After**
LUT	533	544
Flipflop	591	591
Power	3.234 W	3.159 W
Delay	11.001 ns	10.887 ns

Table 4.1. Pre Optimization and post optimization

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.0.1 Conclusion

In conclusion the pursuit of an optimized FPGA implementation of the SKINNY-AEAD block cipher for lightweight applications is well underway. The decision to focus on FPGA implementation reflects a pragmatic approach, capitalizing on the parallelism and adaptability intrinsic to FPGAs to optimize the cipher's efficiency. While certain aspects remain unresolved, the project's progress underscores its potential significance in the realm of lightweight cryptography. Moving forward, emphasis should center on further refining the implementation to address identified challenges and enhance overall efficiency. This ongoing work holds considerable promise, particularly in fortifying the security of lightweight devices, where resource limitations are critical considerations.

5.0.2 Future Scope

As technology evolves, the demand for lightweight cryptographic solutions, like those for IoT devices, wearables, and resource-constrained environments, grows. Future development in this area focuses on enhancing efficiency and security in FPGA-based implementations. This entails refining the cryptographic algorithm and optimizing its hardware realization on FPGAs through innovative architectural designs and circuit-level refinements. Ensuring adaptability to changing demands is crucial, including integration with emerging technologies and alignment with evolving security standards. Future research may also explore strategies for optimizing power consumption and real-time performance. Overall, the future outlook involves

a comprehensive approach to advancing both the core cryptographic algorithm and its hardware implementation to meet the evolving needs of lightweight applications.

REFERENCES

[1] Christof Beierle, Jeremy Jean, Stefan Kolbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, Siang Meng Sim(2021)"Skinny AEAD and Skinny HASH", Status Report on the First Round of the NIST Lightweight Cryptography Standardization Process

[2] N. Ahmad, R. Hasan and W. M. Jubadi, "Design of AES S-box using combinational logic optimization," 2010 IEEE Symposium on Industrial Electronics and Applications (ISIEA), Penang, Malaysia, 2010

[3] Shreenivas Pai N, Raghuram S, Chennakrishna M and A. S. V. Karthik, "Logic optimization of AES S-Box," 2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICACDOT), Pune, India, 2016

[4] Beierle, Christof, et al. "The SKINNY family of block ciphers and its low-latency variant MANTIS." Advances in Cryptology–CRYPTO 2016: 36th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 14-18, 2016, Proceedings, Part II 36. Springer Berlin Heidelberg, 2016.

[5] Vafaei, Navid, Sayandeep Saha, Nasour Bagheri, and Debdeep Mukhopadhyay. "Fault attack on SKINNY cipher." Journal of Hardware and Systems Security 4 (2020): 277-296.

[6] Zhang, Pei, and Wenying Zhang. "Differential cryptanalysis on block cipher skinny with MILP program." Security and Communication Networks 2018 (2018).

[7] Zhang, P. and Zhang, W., 2018. Differential cryptanalysis on block cipher skinny with MILP program. Security and Communication Networks, 2018.

[8] Vafaei, Navid, Nasour Bagheri, Sayandeep Saha, and Debdeep Mukhopadhyay. "Differential fault attack on SKINNY block cipher." In Security, Privacy, and Applied Cryptography Engineering: 8th International Conference,

SPACE 2018, Kanpur, India, December 15-19, 2018, Proceedings 8, pp. 177-197. Springer International Publishing, 2018.

[9] Zheng Y, Wu W. Biclique attack of block cipher SKINNY. InInternational Conference on Information Security and Cryptology 2016 Nov 4 (pp. 3-17). Cham: Springer International Publishing.

[10] Feng, Jingya, and Lang Li. ”SCENERY: a lightweight block cipher based on Feistel structure.” Frontiers of Computer Science 16.3 (2022): 163813.

APPENDIX

VHDL CODE

Sbox Code

```
library ieee;

use ieee.std_logic_1164.all;

entity sbox is

port (

STATE : in std_logic_vector(7 downto 0);

SUB_OUT : inout std_logic_vector(7 downto 0)

);

end sbox;

architecture behavioral of sbox is

signal in_mod, selx: std_logic_vector(3 downto 0);

signal in_mux1,in_mux2,in_mux3,in_mux4,in_mux5,in_mux6,in_mux7,in_mux8,
in_mux9,in_mux10 ,in_mux11,in_mux12,in_mux13,in_mux14,in_mux15,in_mux16 :

std_logic_vector(7 downto 0);

component m1 is

Port ( e,f,g,h : in STD_LOGIC;

k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);

end component m1;
```

```
component m2 is
Port ( e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m2;

component m3 is
Port ( e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m3;

component m4 is
Port ( e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m4;

component m5 is
Port ( e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m5;

component m6 is
Port (  e,f,g,h: in STD_LOGIC;
 k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m6;

component m7 is
```

```
Port (  e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : STD_LOGIC);
end component m7;

component m8 is
Port (  e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m8;

component m9 is
Port ( e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m9;

component m10 is
Port ( e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m10;

component m11 is
Port (  e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m11;

component m12 is
Port (  e,f,g,h: in STD_LOGIC;
```

```
k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m12;

component m13 is
Port (  e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0: out STD_LOGIC);
end component m13;

component m14 is
Port (  e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m14;

component m15 is
Port (  e,f,g,h: in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m15;

component m16 is
Port (  e,f,g,h : in STD_LOGIC;
k7,k6,k5,k4,k3,k2,k1,k0 : out STD_LOGIC);
end component m16;

component mux is
Port (Sel : in std_logic_vector(3 downto 0);
```

```
m1,m2,m3,m4,m5,m6,m7,m8,m9,m10,m11,m12,m13,m14,m15,m16 : in

STD_LOGIC_VECTOR (7 downto 0);

T: out std_logic_vector (7 downto 0));

end component mux;

begin

in_mod <= STATE(3) & STATE(2) & STATE(1) & STATE(0);

selx <= STATE(7) & STATE(6) & STATE(5) & STATE(4);

mod1: m1 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux1(7),in_mux1(6),
in_mux1(5),in_mux1(4),in_mux1(3),in_mux1(2),in_mux1(1),in_mux1(0));

mod2: m2 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux2(7),in_mux2(6),
in_mux2(5),in_mux2(4),in_mux2(3),in_mux2(2),in_mux2(1),in_mux2(0));

mod3: m3 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux3(7),in_mux3(6),
in_mux3(5),in_mux3(4),in_mux3(3),in_mux3(2),in_mux3(1),in_mux3(0));

mod4: m4 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux4(7),in_mux4(6),
in_mux4(5),in_mux4(4),in_mux4(3),in_mux4(2),in_mux4(1),in_mux4(0));

mod5: m5 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux5(7),in_mux5(6),
in_mux5(5),in_mux5(4),in_mux5(3),in_mux5(2),in_mux5(1),in_mux5(0));

mod6: m6 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux6(7),in_mux6(6),
in_mux6(5),in_mux6(4),in_mux6(3),in_mux6(2),in_mux6(1),in_mux6(0));

mod7: m7 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux7(7),in_mux7(6),
in_mux7(5),in_mux7(4),in_mux7(3),in_mux7(2),in_mux7(1),in_mux7(0));

mod8: m8 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux8(7),in_mux8(6),
in_mux8(5),in_mux8(4),in_mux8(3),in_mux8(2),in_mux8(1),in_mux8(0));

mod9: m9 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux9(7),in_mux9(6),
in_mux9(5),in_mux9(4),in_mux9(3),in_mux9(2),in_mux9(1),in_mux9(0));

mod10: m10 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux10(7),
in_mux10(6),in_mux10(5),in_mux10(4),in_mux10(3),in_mux10(2),in_mux10(1),
in_mux10(0));

mod11: m11 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux11(7),
in_mux11(6),in_mux11(5),in_mux11(4),in_mux11(3),in_mux11(2),in_mux11(1),
in_mux11(0));

mod12: m12 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux12(7),
in_mux12(6),in_mux12(5),in_mux12(4),in_mux12(3),in_mux12(2),in_mux12(1),
```

```
in_mux12(0));

mod13: m13 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux13(7),
in_mux13(6),in_mux13(5),in_mux13(4),in_mux13(3),in_mux13(2),in_mux13(1),
in_mux13(0));

mod14: m14 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux14(7),
in_mux14(6),in_mux14(5),in_mux14(4),in_mux14(3),in_mux14(2),in_mux14(1),
in_mux14(0));

mod15: m15 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux15(7),
in_mux15(6),in_mux15(5),in_mux15(4),in_mux15(3),in_mux15(2),in_mux15(1),
in_mux15(0));

mod16: m16 Port map(in_mod(3),in_mod(2),in_mod(1),in_mod(0),in_mux16(7),
in_mux16(6),in_mux16(5),in_mux16(4),in_mux16(3),in_mux16(2),in_mux16(1),
in_mux16(0));

mux16: mux Port map(selx,in_mux1,in_mux2,in_mux3,in_mux4,in_mux5,in_mux6,
in_mux7,in_mux8,in_mux9,in_mux10,in_mux11,in_mux12,in_mux13,in_mux14,in_mux15,
in_mux16,SUB_OUT);

end architecture behavioral;
```

Module 1 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m1 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m1;

architecture Behavioral of m1 is

begin

    process (e,f,g,h)

    begin
```

```
        k7 <= '0';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= (g) or (f);

        k0 <= (not g and not h) or (f) or (e and not g);

    end process;

end behavioral;
```

Module 2 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m2 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m2;

architecture Behavioral of m2 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and h) or (f and not h) or (e and not h);

        k6 <=  '0';
```

```
        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and not h) or (not e and f and h) or (e and not h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= (not e and not f and g and not h) or (f and h) or (e and g and h);

        k0 <= (not e and g and h) or (e and f and g) or (not e and not g and not h)
        or (f and h) or (e and not f and not g);

    end process;

end behavioral;
```

Module 3 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m3 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m3;

architecture Behavioral of m3 is

begin

    process (e,f,g,h)

    begin

        k7 <= '1';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);
```

```
        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= '0';

        k0 <= (not e and g and h) or (not e and not g and not h) or
        (e and not f and not g) or (e and f and g);

    end process;

end behavioral;
```

Module 4 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m4 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m4;

architecture Behavioral of m4 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and not h) or (f and h) or (e and h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);
```

```
        k4 <= (not f and h) or (not e and f and not h) or (e and h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= (not e and not f and g and h) or (f and not h) or (e and g and not h);

        k0 <= (not g and not h) or (f and g) or (e and not f and not g);

    end process;

end behavioral;
```

Module 5 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m5 is

    Port (e, f, g, h : in std_logic;

        k7, k6, k5, k4, k3, k2, k1, k0 : out std_logic);

end m5;

architecture Behavioral of m5 is

begin

    process (e, f, g, h)

    begin

        k7 <= (not e and not f and h) or (f and not h) or (e and not h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and not h) or (not e and f and h) or (e and not h);
```

```
        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= g or f;

        k1 <= (not e and not f and not g and not h) or (e and not f and not g and h);

        k0 <= (not e and g and h) or (not e and f and not g and not h) or (e and
        not f and not g and not h) or (e and f and g);

    end process;

end Behavioral;
```

Module 6 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC _STD.ALL;

entity m6 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m6;

architecture Behavioral of m6 is

begin

     process (e,f,g,h)

     begin

        k7 <= '0';

        k6 <= (e) or (not f and not g) or (g and not h) or (f and h);

        k5 <= (e and h) or (not e and not f and not h) or (g and h) or (not e
        and f and not g);

        k4 <= (e);
```

```
        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= (not f and not g);

        k0 <= (not e and g and h) or (not e and f and not g and not h) or
        (e and f and g);

    end process;

end behavioral;
```

Module 7 Code

```
Library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m7 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m7;

architecture Behavioral of m7 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and not g and not h) or (f and h) or (e and h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and h) or (not e and f and not h) or (e and h);
```

```
        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= (not e and not f and not g and h) or (e and not f and not
        g and not h);

        k0 <= (not e and not g and not h) or (not e and g and h) or
        (e and not f and not g and h) or (e and f and g);

    end process;

end behavioral;
```

Module 8 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m8 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m8;

architecture Behavioral of m8 is

begin

    process (e,f,g,h)

    begin

        k7 <= '1';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);
```

```
        k2 <= (not f and g) or (f and not g) or (g and h) or (e and g);

        k1 <= '0';

        k0 <= (not e and g and h) or (not e and not g and not h) or (e and not
        f and not g) or (e and f and g);

    end process;

end behavioral;
```

Module 9 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m9 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m9;

architecture Behavioral of m9 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and h) or (f and not h) or (e and not h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and not h) or (not e and f and h) or (e and not h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);
```

```
        k1 <= (not e and not f and h) or (f and not h) or (not f and not g) or
        (e and not h);

        k0 <= (f and not h) or (f and g);

    end process;

end behavioral;
```

Module 10 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m10 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m10;

architecture Behavioral of m10 is

begin

    process (e,f,g,h)

    begin

        k7 <= '0';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= (not f and not g);
```

```
        k0 <= (not e and f and not g and not h) or (not e and g and h) or (e and
        not f and g);

    end process;

end behavioral;
```

Module 11 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m11 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m11;

architecture Behavioral of m11 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and not h) or (f and h) or (e and h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and h) or (not e and f and not h) or (e and h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= (not e and not f and not h) or (f and h) or (not f and not g) or
        (e and h);
```

```
        k0 <= (not e and not f and g) or (not e and f and not g) or (f and h)
        or (e and f and g);

    end process;

end behavioral;
```

Module 12 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m12 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m12;

architecture Behavioral of m12 is

begin

    process (e,f,g,h)

    begin

        k7 <= '1';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (not f and not g);

        k1 <= '1';
```

```
        k0 <= (f);

    end process;

end behavioral;
```

Module 13 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m13 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m13;

architecture Behavioral of m13 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and h) or (f and not h) or (e and not h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and not h) or (not e and f and h) or (e and not h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= (not e and h) or (g) or (f) or (e and not h);

        k0 <= (not e and not g and not h) or (f) or (e and not g and h);
```

```
    end process;

end behavioral;
```

Module 14 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m14 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m14;

architecture Behavioral of m14 is

begin

    process (e,f,g,h)

    begin

        k7 <= '0';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= (g) or (f);

        k0 <= (not g and not h) or (f) or (e and not g);

    end process;
```

```
end behavioral;
```

Module 15 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m15 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m15;

architecture Behavioral of m15 is

begin

    process (e,f,g,h)

    begin

        k7 <= (not e and not f and not h) or (f and h) or (e and h);

        k6 <= '0';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (not f and h) or (not e and f and not h) or (e and h);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= (not e and not h) or (g) or (f) or (e and h);

        k0 <= (f) or (e and not g and not h);

    end process;

end behavioral;
```

Module 16 Code

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity m16 is

    Port (e,f,g,h : in std_logic;

        k7,k6,k5,k4,k3,k2,k1,k0 : out std_logic);

end m16;

architecture Behavioral of m16 is

begin

    process (e,f,g,h)

    begin

        k7 <= '1';

        k6 <= '1';

        k5 <= (not e and not f and not h) or (f and h) or (e and h);

        k4 <= (e);

        k3 <= (not e and not f and not g and h) or (not f and g and not h) or
        (not e and f and not g and not h) or (f and g and h) or (e and g);

        k2 <= (g) or (f);

        k1 <= '1';

        k0 <= (f);

    end process;

end behavioral;
```

MUX

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity mux is
Port (Sel : in std_logic_vector(3 downto 0);
m1,m2,m3,m4,m5,m6,m7,m8,m9,m10,m11,m12,m13,m14,m15,m16 : in
STD_LOGIC_VECTOR (7 downto 0);
T : out std_logic_vector (7 downto 0));
end mux;
architecture Behavioral of mux is
begin
process (Sel, m1,m2,m3,m4,m5,m6,m7,m8,m9,m10,m11,m12,m13,m14,m15,m16)
begin
if (Sel = "0000") then T<= m1;
elsif (Sel = "0001") then T<= m2;
elsif (Sel = "0010") then T<= m3;
elsif (Sel = "0011") then T<= m4;
elsif (Sel = "0100") then T<= m5;
elsif (Sel = "0101") then T<= m6;
elsif (Sel = "0110") then T<= m7;
elsif (Sel = "0111") then T<= m8;
elsif (Sel = "1000") then T<= m9;
elsif (Sel = "1001") then T<= m10;
elsif (Sel = "1010") then T<= m11;
```

```
elsif (Sel = "1011") then T<= m12;

elsif (Sel = "1100") then T<= m13;

elsif (Sel = "1101") then T<= m14;

elsif (Sel = "1110") then T<= m15;

else T<= m16;

end if ;

end process ;

end Behavioral;
```

APPENDIX

DATA SHEETS

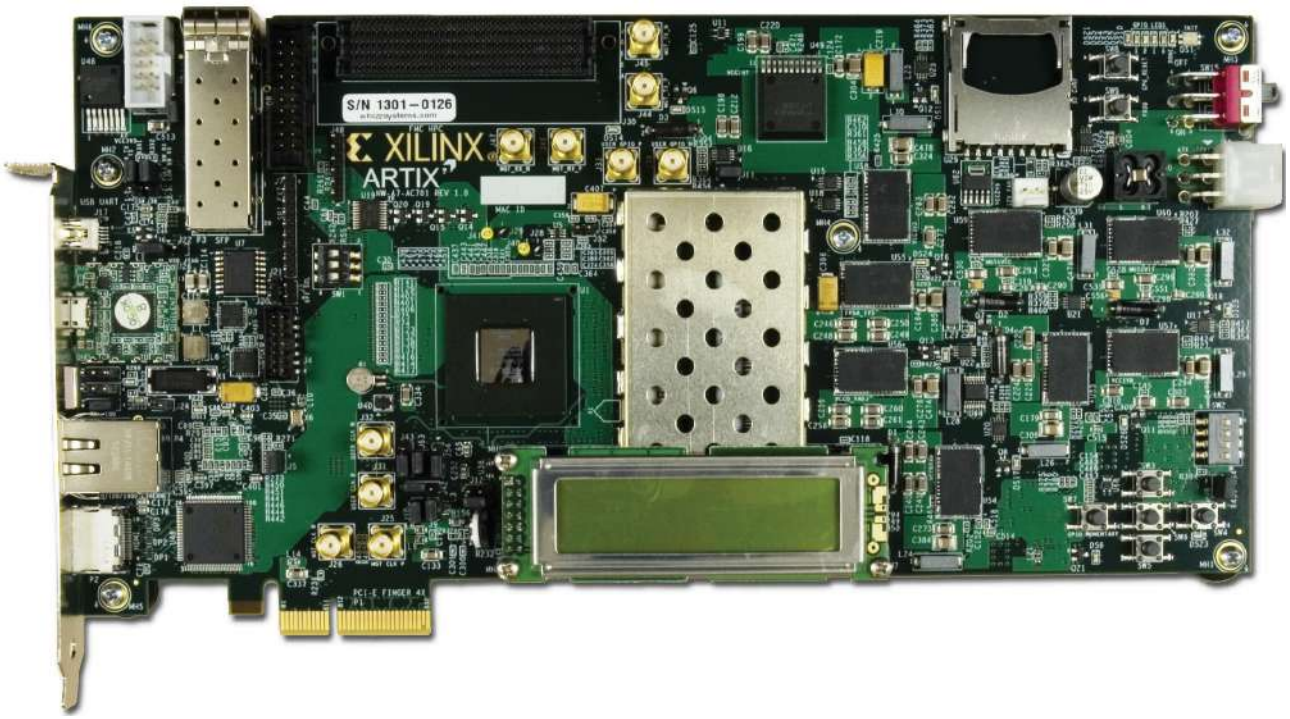

Fig. .1. AC701 Board Components

Callout	Reference Designator	Component Description	Notes
1	U1	Artix-7 FPGA	Xilinx XC7A200T-2FBG676C
2	J1	DDR3 SODIMM socket with memory	Micron MT8JT12864HZ-1G6G1
3	U7	Quad SPI flash memory	Micron N25Q256A13ESF40G
4	U29	SD card interface connector	Molex 67840-8001
5	U26	USB-JTAG module	Digilent USB JTAG module (with micro-B receptacle)

Callout	Reference Designator	Component Description	Notes
6	U51	System clock source (back side of board)	SiTime SIT9102AI-243N25E200.0000
7	U34	Programmable user clock source 10 MHz-810 MHz (back side of board)	Silicon Labs SI570BAB000544DG (default 156.250 MHz)
8	J31, J32	SMA user clock input	Rosenberger 32K10K-400L5
9	J25, J26	SMA GTP reference clock input	Rosenberger 32K10K-400L5
10	U24	Jitter attenuated clock (back side of board)	Silicon Labs SI5324-C-GM
11	U1	GTP transceivers	Embedded within FPGA U1
12	P1	PCI Express® edge connector	4-lane card edge connector
13	P3	SFP/SFP+ connector	Molex 74441-0010
14	U12	10/100/1000 tri-speed Ethernet PHY	Marvell 88E1116RA0-NNC1C000
15	U2	GTP transceiver clock generator 125 MHz	ICS ICS84402IAGI-01LF
16	J17, U44	USB-to-UART bridge (back side of board) and mini-B receptacle (front side of board)	Silicon Labs CP2103GM
17	P2, U48	HDMI video connector and device	Molex 500254-1927, Analog Devices ADV7511KSTZ-P
18	J23	LCD character display connector	2 x 7 0.1 in male pin header
19	U52	I2C bus switch (back side of board)	TI PCA9548ARGER
20	DS11–DS13	Ethernet PHY status LEDs, green	Lumex SML-LX0603GW
21	SW3 – SW7	User pushbuttons E-switch	E-Switch TL3301EF100QG
22	SW2	GPIO DIP switch, 4-pole	C and K SDA04H1SBD
23	DS2–DS5	User GPIO LEDs, greeny	Lumex SML-LX0603GW
24	SW10	User rotary switch	Panasonic EVQ-WK4001
25	J33, J34	SMA user GPIO	Rosenberger 32K10K-400L5
26	SW15	Power on/off slide switch	CandK 1201M2S3AQE2
27	SW9	FPGA PROG B pushbutton switch (active-Low)	E-Switch TL3301EF100QG
28	SW1	Configuration mode DIP switch, 3-pole	CandK SDA03H1SBD

Callout	Reference Designator	Component Description	Notes
29	J30	FMC HPC connector	Samtec ASP 134486 01
30	U8, U9, U49, U53-U60	Power management (voltage regulators front side of board, controllers back side of board)	TI UCD90120ARGC controllers in conjunction with various regulators
31	J19	XADC header	2X10 0.1 inch male header
32	J44, J45, J46, J47	MGT transmit, receive SMA pairs	Rosenberger 32K10K-400L5
33	J2	PMBus connector	Assmann AWHW10G-0202-T-R 1
34	J49	6 pin Molex mini-fit Jr. connector for 12Vy	Molex 39-30-1060
35	U3, U4	GTP transceiver clock multiplexers	Micrel SY89544UMG

Table .1. AC701 Board Component Descriptions

Configuration Mode	SW1 DIP switchSettings (M[2:0])	BusWidth	CCLKDirection
Master SPI flash memory	001	x1, x2, x4	Output
JTAG	101	x1	Not applicable

Table .2. AC701 Board FPGA Configuration Modes